

Register API

Documentation



POST /api/register — Stylish E-commerce Platform

Controller · Route · Service Pattern

Node.js · Express · bcrypt · jsonwebtoken

PROJECT	Stylish Shoes Online Store
MODULE	Authentication — Registration
STACK	Node.js 22 · Express 4 · bcrypt 6 · JWT 9
DATA	data/json/users.json (10 seed users + live writes)
DATE	May 2026

GenAI Prompts

The two prompts below were used to generate the mock database and the authentication route. They are recorded here verbatim so the generation steps are reproducible and auditable.

PROMPT 1 — Database Preparation

Generate a JSON file of 10 registered users with username (email), password (hash with MD5), first name, and date of registration. Also, give me the list of paired email and plaintext password for login testing.

■ *Implementation note: MD5 is not suitable for password storage. bcrypt was used instead because it is intentionally slow and embeds a unique salt in every hash, preventing rainbow-table and brute-force attacks.*

PROMPT 2 — API Route Generation

Act as a Security Engineer. Write a Node.js/Express POST route for /api/login. It should receive an email and a password. If the email is not in my database, return 401 Unauthorized status. If it exists, use a library like bcrypt to compare the submitted password with a hashed one in my database. If it matches, use jsonwebtoken to sign a token containing the user's ID. Return the token with a 200 status.

API Contract Table

All three endpoints exposed by the Stylish backend. Each row defines the method, path, request shape, and every possible response status with its payload schema.

2.1 Endpoint Overview

Method	Route	Auth Required	Purpose
GET	/api/products	No	List products; optional ?category= filter
POST	/api/login	No	Authenticate user, receive JWT
POST	/api/register	No	Create new user account

2.2 POST /api/register — Full Contract

Request Body (Content-Type: application/json)

Field	Type	Required	Validation Rules
firstName	string	Yes	Non-empty · max 50 chars
email	string	Yes	Valid email shape · lowercased on arrival
password	string	Yes	Min 8 chars · 1 uppercase · 1 digit · 1 special char
confirmPassword	string	Yes	Must match `password` exactly

Response Codes

Status	Condition	Response Body
201 Created	All fields valid, email not taken, user written to DB	{ "success": true, "message": "Welcome, Noon!", "user": { "id", "firstName", "email", "registeredAt" } }
400 Bad Request	Missing field / type mismatch / weak password / passwords don't match	{ "success": false, "error": "`password` and `confirmPassword` do not match." }
409 Conflict	Email already exists in users.json	{ "success": false, "error": "An account with that email address already exists." }
500 Internal Server Error	JSON write failure or unexpected runtime error	{ "success": false, "error": "Failed to create account due to a server error." }

2.3 POST /api/login — Abbreviated Contract

Status	Condition	Response Body
200 OK	Email found in DB, bcrypt.compare succeeds	{ "success": true, "token": "eyJ...", "user": { "id", "firstName", "email" } }
400 Bad Request	Missing or malformed email / empty password	{ "success": false, "error": "\"email\" is required." }
401 Unauthorized	Email not found OR password wrong (intentionally same message)	{ "success": false, "error": "Invalid email or password." }
500 Internal Server Error	File I/O or JWT signing failure	{ "success": false, "error": "An internal server error occurred." }

Architecture Diagrams

3.1 Controller–Route–Service File Structure

Layer	File	Responsibility
Route	server/routes/register.js	Declares POST / pipeline: [Gatekeeper → Controller]
Route	server/routes/auth.js	Declares POST / pipeline: [Gatekeeper → Controller]
Middleware	server/middleware/validateRegisterBody.js	Gatekeeper: shape, types, password strength, confirmPassword match
Middleware	server/middleware/validateLoginBody.js	Gatekeeper: required fields, email format
Controller	server/controllers/registerController.js	HTTP bridge: extracts body, calls service, maps errors to status codes
Controller	server/controllers/authController.js	HTTP bridge for login: maps INVALID_CREDENTIALS → 401
Service	server/services/registerService.js	Duplicate check → bcrypt.hash → JSON write → return safe user
Service	server/services/authService.js	Lookup → constant-time bcrypt.compare → jwt.sign
Data	data/json/users.json	Mock database: 10 seed users (bcrypt hashes) + live appends
Entry	server/app.js	Mounts all routes under /api/*; global middleware
Entry	server/server.js	Binds to PORT 3000; imports app.js

3.2 POST /api/register — UML Sequence Diagram

Full UML sequence diagram showing the request lifecycle. Gold-circled numbers mark each message step. Dashed lifelines connect actors. Alt frames show the Gatekeeper 400 short-circuit and the 409 duplicate-email branch. Dashed arrows are return/response flows.

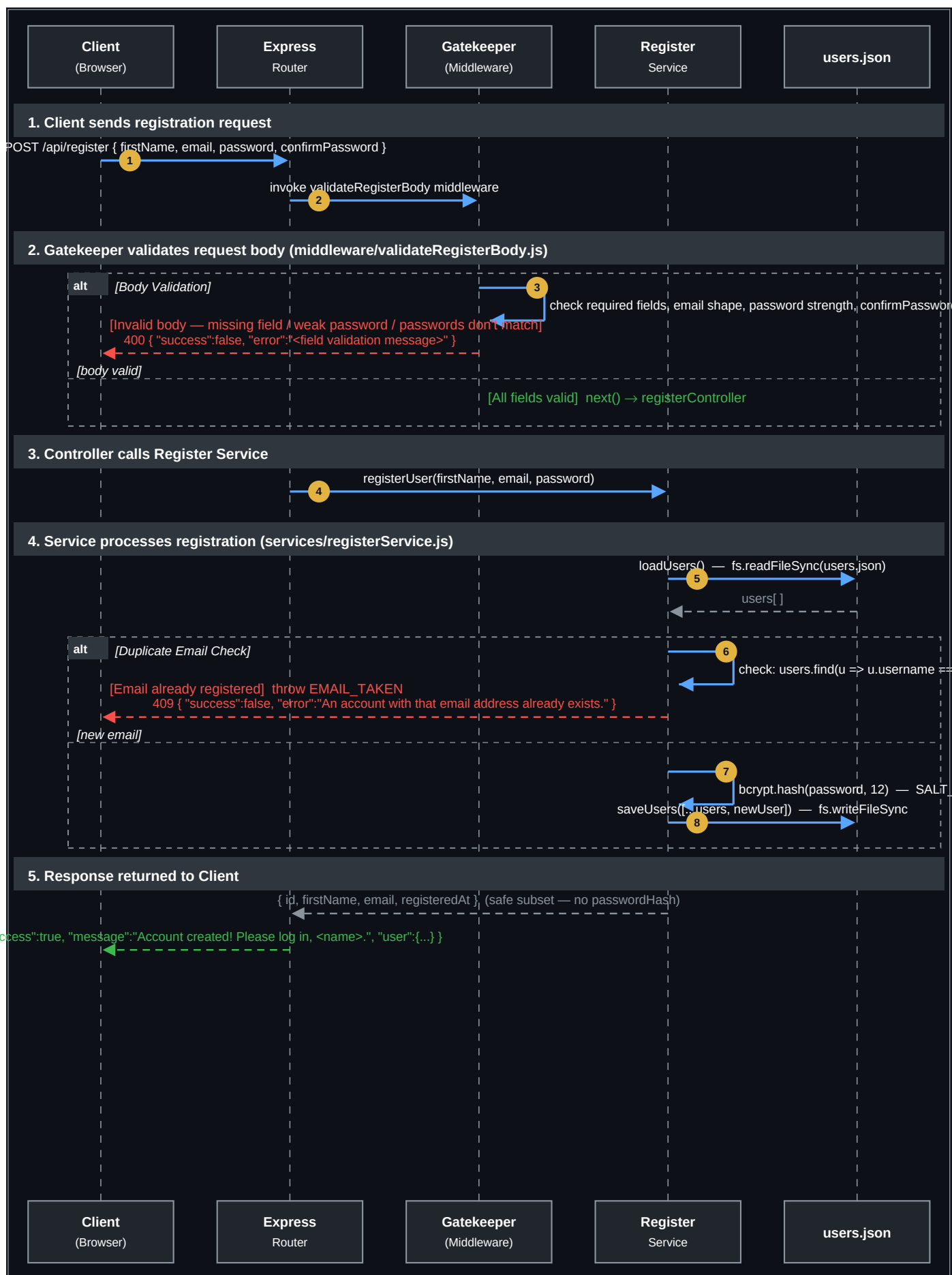


Fig 1. `POST /api/register` sequence — Client → Router → Gatekeeper → Register Service → `users.json`. Alt frames cover body-validation failure (400), duplicate-email conflict (409), and the success path (201).

3.3 Security Design Decisions

Decision	Reason
bcrypt (SALT_ROUNDS=12) instead of MD5	MD5 is a fast digest crackable in milliseconds with a GPU. bcrypt is intentionally slow; cost factor 12 takes ~250 ms per hash, making brute-force impractical.
Generic 401 message 'Invalid email or password.'	Returning different messages for 'email not found' vs 'wrong password' allows user-enumeration. A single message leaks nothing.
Constant-time <code>bcrypt.compare</code> for unknown emails	Without this, an attacker can detect valid emails via response-time differences (~0 ms vs ~250 ms). Comparing against a dummy hash equalises timing.
JWT payload contains only <code>id</code>, <code>firstName</code>, <code>email</code>	JWTs are Base64-decodable by anyone. Sensitive data (<code>passwordHash</code> , addresses, payment info) must never appear in the payload.
JWT_SECRET from <code>process.env</code>	Hardcoding secrets in source code exposes them in git history and build artefacts. Environment variables keep them out of the codebase.
<code>confirmPassword</code> checked in Gatekeeper, not service	Structural validation belongs at the boundary. The service should never receive mismatched passwords — it assumes inputs are pre-validated.